ExpenseSense: Privacy-Preserving On-Device Personal Finance Tool Calling with Sub-2B LLMs

Sajag Swami **Yash Pandey**
Independent Researchers
sajagswami@gmail.com ya.pandey@outlook.com

Abstract

Personal finance data - detailed records of when, where, and how a user spends money - is among the most sensitive information individuals possess, making cloud-based LLM inference undesirable for many users. Yet natural-language personal finance assistants currently require transmitting these queries to cloud-hosted LLMs, exposing users to data retention and third-party data flows. We introduce **ExpenseSense**, a personal finance Q&A assistant that runs entirely on-device, ensuring that no financial records, query text, or model outputs leave the user's device. To evaluate whether this privacy architecture is practically viable, we benchmark on-device tool calling using a 115-question test suite across six sub-2B quantized SLMs (0.8B-2B), comparing single-agent (joint routing and parameter extraction) against dual-agent (router-specialist) architectures. The dual-agent advantage is metric-specific: while optimised single-agent Qwen3.5-2B reaches the highest Task Accuracy (92.0%), dual-agent Qwen3.5-2B achieves 77.6% strict Correct Ratio (+17.4 pp over single-agent) while reducing average latency by 48%, and dual-agent Qwen3.5-0.8B reaches 89.1% Task Accuracy in 3.0 s with 1.2 GB RAM. Critically, dual-agent architectures shift errors from spurious parameters to safer, missing parameters, reducing the risk of silent incorrect outputs. Our results demonstrate that private, high-quality financial analytics is achievable on consumer hardware today, with no cloud dependency.

1 Introduction

Large language models (LLMs) have rapidly advanced tool-augmented AI (Schick et al., 2023; Patil et al., 2023; Qin et al., 2024), enabling agents to invoke structured APIs from natural language. However, research primarily targets large cloud-hosted models which is problematic for personal finance where local inference is necessary for privacy. Expense records reveal sensitive behavioral patterns (e.g., health-related purchases). Users expect natural language interfaces (e.g., asking "how much did I spend on food last month?") that map to structured queries over local data (Li et al., 2024). This creates an underexplored problem: can *sub-2B quantized models* running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial language, and in doing so, serve as practical privacy infrastructure for real users?

Existing benchmarks (Qin et al., 2024; Zhu et al., 2026; Cao et al., 2026) focus primarily on multi-step trajectories across financial APIs for cloud-scale models, which are prohibitively slow for local deployment. Conversely, on-device models like Octopus v2 (Chen and Li, 2024) and Hammer (Lin et al., 2024) demonstrate the potential of smaller models but have not been evaluated on domain-specific financial schemas or under consumer-hardware latency constraints.

We address three research questions (**RQ**):

- **RQ1**: Does on-device inference with sub-2B SLMs provide a practically viable privacy protection for personal finance tool use-i.e., can users obtain accurate financial analytics without surrendering query data to cloud providers?
- **RQ2**: Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy gains over a joint single-agent architecture?
- **RQ3**: What are the primary failure modes across architectures, and do they differ in the human-facing harms they produce?

This paper introduces **ExpenseSense**, with the following contributions:

1. Empirical **on-device deployment guidelines for privacy**: Demonstrating that sub-2B

SLMs on consumer hardware (e.g., Qwen3.5-0.8B in dual mode achieving 89.1% Task Accuracy in 3.0 s with 1.2 GB RAM) provide a practical privacy protection, eliminating cloud data transmission for personal finance.

2. A comparative **architectural evaluation** of single- vs. dual-agent setups across six sub-2B models using a **7-feature complexity scoring model** (L1–L3 tiers) to systematically benchmark performance under realistic, informal query constraints.

3. A fine-grained **error taxonomy** of 7 categories showing that dual-agent decomposition consolidates errors into safer, predictable failure modes—shifting from dangerous spurious parameter additions (`PARAM_EXTRA`) toward visible omissions (`PARAM_MISSING`).

2 On-Device Inference as Privacy Protection for Real Users

The deployment of LLMs in personal finance exposes a structural tension: the same conversational interface that makes financial analytics accessible to users also makes their sensitive behavioral data accessible to cloud providers. When a user types "how much did I spend on therapy last month?" into a cloud-hosted assistant, they are not merely querying a database-they transmit evidence of a health condition, a temporal pattern, and financial commitment to a third-party infrastructure they did not choose and cannot audit. The query is a disclosure.

Staab et al. (2024) demonstrated that LLMs can infer sensitive personal attributes-including location, income, and demographic characteristics-from seemingly innocuous texts, and Yao et al. (2024) catalog privacy risks including data leakage, memorization, and inference attacks. The implication for personal financial assistants is direct: even if the database never leaves the device, routing natural-language queries through cloud LLMs leaks behavioral signals that can be retained, aggregated, and exploited beyond the user's control or knowledge.

ExpenseSense addresses this threat by adopting an *on-device, privacy-first* architecture. By leveraging sub-2B quantized models that run efficiently on consumer hardware (tested on an Apple M1 with 8 GB unified memory), all user data, query text, and model outputs remain strictly local. No financial records, queries or inferences leave the device.

The local-first design preserves *contextual integrity* (Nissenbaum, 2004) by ensuring financial queries and data remain within their original context.

The practical threat model covers three risks: cloud data retention, query-level inference attacks (Staab et al., 2024), and third-party exposure. On-device inference eliminates all three by ensuring queries never leave for external infrastructure.

The key challenge is that on-device inference with sub-2B models has historically been unreliable for structured tool calling (Kavathekar et al., 2025). Our work demonstrates that architectural decomposition-separating intent classification from parameter extraction-bridges this reliability gap, making private on-device inference not merely possible but practical.

3 Related Work

Tool-Augmented Language Models. Toolformer (Schick et al., 2023) demonstrated self-supervised tool-use learning; Gorilla (Patil et al., 2023) and ToolLLM (Qin et al., 2024) scaled to thousands of real-world APIs. HuggingGPT (Shen et al., 2023) and ToolAlpaca (Tang et al., 2023) explored multi-step orchestration. APIGen (Liu et al., 2024b) introduced a verified pipeline for generating function-calling training data, showcasing that 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. These approaches rely on cloud-hosted models ($\geq$7B), leaving the privacy-constrained on-device setting largely unaddressed.

Function-Calling Evaluation. The Berkeley Function Calling Leaderboard (Patil et al., 2025) established a benchmark using AST substring matching and execution-based validation. Kavathekar et al. (2025) proposed an evaluation suite (FSP, ACS, AVC, Task Accuracy, and Correct Ratio) to evaluate small model function calling; we adopt these metrics directly. CallNavi (Song et al., 2025) proposed a two-stage router+specialist approach-similar to our dual-agent design-but evaluates models $\geq$7B on general-purpose APIs. In contrast, we focus on sub-2B models for domain-specific, single-turn personal finance related tasks.

On-Device Function Calling. MobileLLM (Liu et al., 2024a), Phi-3 (Abdin et al., 2024), and related surveys (Xu et al., 2024) have explored sub

4B parameter models for mobile deployment. Octopus v2 (Chen and Li, 2024) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer (Lin et al., 2024) introduced function masking to reduce overfitting to function names, addressing an issue similar to the category normalisation errors we observe. Apple Intelligence (Gunter et al., 2024) validated on-device tool calling at scale with a ∼3B model, demonstrating the viability of the paradigm. Kavathekar et al. (2025) found that structured output compliance is the primary bottleneck for SLMs.

Financial NLP and Privacy. FinQA (Chen et al., 2021) and FinGPT (Yang et al., 2023) target financial numerical reasoning and generation, but focus on text generation rather than tool calling. FinMCP-Bench (Zhu et al., 2026) and FinTrace (Cao et al., 2026) evaluate LLM agents for financial tool use, focusing primarily on multi-step, long-horizon trajectories across cloud-scale models. Personal finance assistants (Li et al., 2024) require strict privacy, making local execution crucial.

Human-Centered Privacy for Language Models. A growing body of work has seen privacy not as a technical property of models but as a human-centered concern shaped by user expectations and social context. Nissenbaum (2004)'s contextual integrity framework can be applied to conversational AI to show that data flows violate privacy when they cross contextual norms, even when no explicit PII is transmitted. Staab et al. (2024) demonstrated that LLMs act as inference engines, reconstructing sensitive personal attributes from ordinary text-a threat particularly acute when personal finance queries are routed through cloud providers. Yao et al. (2024) surveys the resulting landscape of data leakage, memorization, and inference risks in deployed LLM systems. Our work operationalises these concerns at the system level: rather than studying how cloud LLMs violate privacy, we build and evaluate an architecture that eliminates the exposure pathway entirely, keeping all query data local to the user's device.

4 System Design

4.1 Tool Inventory

ExpenseSense exposes five analytical tools over a user's expense database. Table 1 summarises each tool and its parameter space. These tools span core personal finance use cases: time series analysis, categorical breakdown, period comparison, aggregation, and ranked retrieval.

Tool	ID	Key Params
`plot_time_series`	1	category, year/month, ranges, months, ignore_rent
`plot_distribution`	2	category, year/month/day, ranges, months, ignore_rent
`plot_comparison_bars`	3	category, y1/m1, y2/m2, sm/em/ey, ignore_rent
`calculate_total`	4	category, remarks, year/month/day, months, ignore_rent
`get_top_expenses`	5	n, category, year/month/day, months, min_amount, ignore_rent

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

4.2 Architectural Configurations

We evaluate two local architectures for mapping natural language user queries to tool calls.

Single-Agent Pipeline. The single-agent pipeline uses a single prompt with all five tool definitions, parameter schemas, and joint few-shot examples. The model must select the tool and extract parameters in a single pass, outputting a JSON object containing the tool ID (1–5) and arguments for the chosen tool. The prompt includes structured output rules, date-handling instructions, and 73 tool-specific examples aggregated across all 5 tools. Prompt length: ≈5,500 tokens (context window inclusive).

Dual-Agent Pipeline. The task is decomposed into two sequential inference calls:

1. **Router**: A focused intent-classification prompt with 19 few-shot examples and explicit disambiguation rules (e.g., "compare" requires two distinct time periods) elicits a single digit (1–5).

2. **Specialist**: The predicted tool ID selects a tool-specific prompt containing 13–16 examples for that *one* tool, along with detailed parameter definitions and date-handling rules.

The router sees ≈1,000 tokens; the specialist sees ≈1,500–2,000 tokens with deeper, more directly relevant supervision. This simplifies the task for each model call and provides more relevant in-context examples, though it requires two sequential inference passes. Combined prompt tokens: ≈2,700 per query.

Task Group	Cases	Avg Params	Avg C
Time Series	23	2.9	7.0
Distribution	21	2.5	6.4
Comparison	28	4.4	8.8
Calculate Total	22	2.4	6.3
Top Expenses	21	3.2	6.7
Total	**115**	3.2	7.1

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

4.3 Post-Hoc Parameter Validation

A post-processing validation layer corrects category names deterministically without additional model calls:

1. **Exact match** against the known vocabulary.
2. **Morphological normalisation**: `groceries`→`grocery`, `cafés`→`cafe`.
3. **Edit-distance matching** via difflib (cutoff ≥ 0.80).
4. **Token overlap scoring** (Jaccard; cutoff ≥ 0.50).

All results report both raw (pre-validation) and validated (post-validation) metrics.

5 Benchmark Design

5.1 Test Suite

The benchmark contains 115 test cases across five task groups (Table 2). Queries are written in natural and informal language: abbreviations (*"compare electricity '24 vs '25"*), colloquialisms (*"can ya tell me"*), misspellings (*"distribuution"*), and domain-specific terms (*"combinis"*, *"nomikais"*, *"shinkansen"*). This mimics the Japan-based personal finance context of the target application and tests robustness to spelling variations, colloquialisms, and cultural terms.

Benchmark Construction. The 115 test cases were manually authored by a developer based on daily expense tracking patterns across 30+ categories (including terms like "combini meals" and "nomikai"). Queries range from simple totals to complex cross-year range comparisons, paired with deterministic ground-truth tool calls. Cases cover all five tools with varied linguistic styles (abbreviations, misspellings) and 1–10 parameters per query. Single-author creation is acknowledged as a limitation.

5.2 Complexity Scoring

Each test case is assigned a difficulty score:

$$C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

where n_p = number of expected parameters; $d_{\text{date}} \in \{0\text{–}4\}$ scores date specification complexity; $d_{\text{cat}} \in \{0\text{–}2\}$ is the category normalisation distance; $d_{\text{rel}} \in \{0, 1\}$ flags relative time expressions; $d_{\text{multi}} \in \{0\text{–}2\}$ counts compound parameter groups; $d_{\text{abbr}} \in \{0, 1\}$ flags abbreviated dates; and $d_{\text{hol}} \in \{0, 2\}$ flags calendar knowledge requirements. Scores are binned into three tiers: **L1** ($C \leq 4$, 24 cases), **L2** ($5 \leq C \leq 7$, 45 cases), **L3** ($C \geq 8$, 46 cases).

5.3 Evaluation Metrics

We adopt the metric suite of Kavathekar et al. (2025):

- **FSP** (Function Selection Precision): Binary indicator of correct tool prediction.
- **ACS** (Argument Completeness Score): F_1 on argument *name* overlap.
- **AVC** (Argument Value Correctness): Fraction of correctly named arguments whose values also match.
- **Task Accuracy**: F_1 on the combined set $\{\text{tool}\} \cup \{k{=}v\}$, jointly capturing tool selection and parameter correctness.
- **Correct Ratio (CR)**: Strict binary indicator—1.0 only when Task Acc $= 1.0$, i.e. the entire function call is perfectly correct.

All metrics are averaged across 5 repetitions per test case (temperature $= 0.1$) with 95% bootstrap confidence intervals (Efron, 1979) ($n = 2000$ resamples).

5.4 Error Taxonomy

Each prediction is classified into one of seven mutually exclusive error types: `CORRECT`, `TOOL_WRONG`, `PARSE_FAILURE`, `PARAM_MISSING`, `PARAM_VALUE_WRONG`, `PARAM_EXTRA`, `FORMAT_VIOLATION`. This taxonomy helps diagnose specific failures over just aggregate accuracy, which is crucial for evaluating the safety profile of on-device financial assistants with respect to real-user harm.

Model	Family	Params	RAM
EXAONE-4.0-1.2B	EXAONE-4.0 (2025)	1.2B	2.0 GB
Gemma-3-1B-it	Gemma-3 (2025)	1B	1.6 GB
LFM2.5-1.2B	LFM2.5 (2026)	1.2B	1.4 GB
MiniCPM5-1B	MiniCPM5 (2026)	1B	1.5 GB
Qwen3.5-0.8B	Qwen3.5 (2026)	0.8B	1.3 GB
Qwen3.5-2B	Qwen3.5 (2026)	2B	2.1 GB

Table 3: Models evaluated. All use Q8_0 GGUF quantization via llama.cpp. RAM is average peak RSS.

6 Experimental Setup

6.1 Models

All six models (Table 3) use Q8_0 GGUF quantization and run via llama.cpp (Gerganov, 2023) with full GPU offload on Apple Metal.

6.2 Hardware and Inference Protocol

Experiments run on an Apple M1 (8-core CPU, 8-core GPU, 8 GB unified memory). Parameters: temperature $= 0.1$, min_p $= 0.05$, n_ctx $= 8192$, stop tokens `["\n\n", "```"]`. Each model is warmed up through all six prompt templates (one router + five specialists) before evaluation begins. Each of the 115 test cases is run 5 times per model per architecture, yielding $115 \times 5 \times 6 \times 2 = 6{,}900$ total observations.

7 Results

7.1 Overall Performance

Table 4 presents the full results. Figure 2 shows the accuracy-latency Pareto frontier and the error taxonomy breakdown.

Key findings (RQ1, RQ2). **(1)** Sub-2B SLMs can reliably call tools to perform personal finance analytics on consumer hardware (**RQ1**), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This demonstrates that cloud reliance can be avoided while keeping all data—queries and results—private to the user's device. **(2)** The dual-agent advantage is metric-specific (**RQ2**): single-agent Qwen3.5-2B achieves the highest Task Accuracy (92.0% vs. 90.4% dual), but dual-agent decomposition is preferred for stricter correctness and safety: *(a)* Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, representing a significant reduction in silent parameter errors that could produce incorrect outputs. *(b)* For smaller models, Qwen3.5-0.8B improves from 79.8% to 89.1% Task Accuracy (+9.3 pp) while nearly halving latency (5.8 s to 3.0 s) in dual mode, making sub-1B models viable for low-memory devices. *(c)* The error taxonomy shift from dangerous `PARAM_EXTRA` errors to safer `PARAM_MISSING` errors (§8) represents a qualitative safety improvement not captured by aggregate accuracy.

7.2 Per-Tool-Group Analysis

Table 5 shows per-group accuracy for the top two dual-mode models. Figure 3 shows the full model × category heatmap. `get_top_expenses` reaches 98.9% accuracy with Qwen3.5-2B due to its simple parameter structure. Conversely, `plot_time_series` is the most difficult task (74.4% for Qwen3.5-2B) due to complex date specifications and relative time expressions (e.g., "past X months") requiring temporal reasoning.

7.3 Complexity Tier Analysis

Figure 4 presents the per-model accuracy breakdown across complexity tiers in dual mode. Task Accuracy decreases from 96% (L1) to 82% (L3) for Qwen3.5-2B, and from 95% to 84% for Qwen3.5-0.8B. L3 queries involve constructs like cross-year range comparisons (*"compare groceries nov 2024 to april 2025 vs nov 2025 to april 2026"*), which require extracting up to 10 parameters with complex temporal constraints.

7.4 Error Taxonomy Analysis

Figure 1 visualises the full error distribution. The error distribution shifts significantly between configurations (**RQ3**).

For Qwen3.5-2B: single-agent produces 22.6% `PARAM_EXTRA` and 7.3% `PARAM_MISSING`; dual-agent reduces `PARAM_EXTRA` to 1.9% while `PARAM_MISSING` remains at 8.0%.

The human-facing implications of this shift are significant. A `PARAM_EXTRA` error is a silent corruption: when a model adds a spurious `months=1` filter to the query "how much did I spend on food?", the system returns an answer- last month's food spending- that looks correct but answers a different question. A user relying on this figure for tax preparation, expense reporting, or budgeting receives a plausible but wrong number, with no indication that the scope was silently restricted. In contrast, a `PARAM_MISSING` error produces a visibly broader result (e.g., all-time food spending when

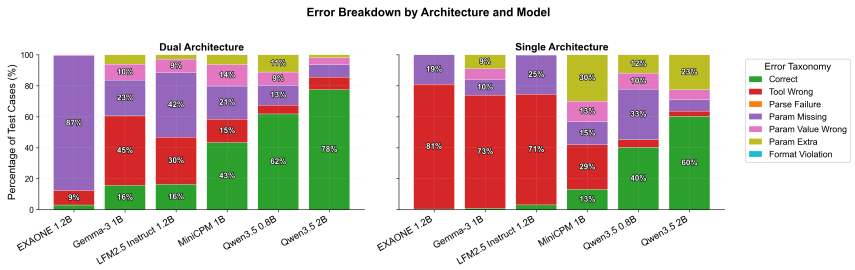
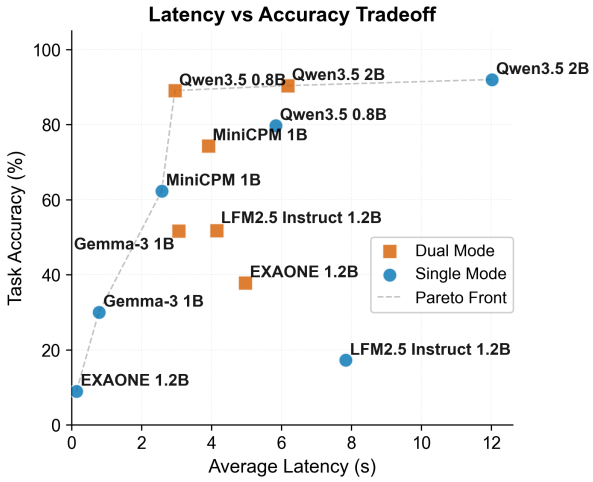
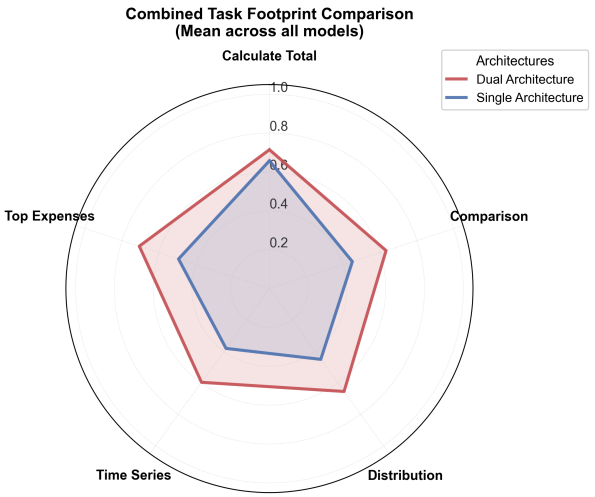


Figure 1: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B (right panel) produces 22.6% `PARAM_EXTRA` errors- spurious parameters that could lead to incorrect computations. Dual-agent (left) eliminates most of these, consolidating residual errors into the more predictable `PARAM_MISSING` category.



(a) Accuracy-latency Pareto frontier. The dual-mode Qwen3.5-0.8B (orange square, ≈3 s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at 4× the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 2: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all task categories.

a month was intended), which is more likely to trigger user scrutiny. The dual-agent architecture's shift from extra-parameter to missing-parameter failures thus represents a qualitative improvement in user-facing safety, not just a numerical accuracy improvement.

For Qwen3.5-0.8B: single-agent produces 32.5% `PARAM_MISSING` and 11.8% `PARAM_EXTRA`; dual-agent reduces these to 12.9% and 11.1% respectively, shifting the model from a high-variance to a more predictable failure profile.

7.5 Token Efficiency

Figure 5 shows that despite using ≈2,700 prompt tokens per query (vs. ≈5,500 for single-agent), the dual-agent architecture achieves equivalent or superior accuracy. The single-agent pipeline gets access to larger contexts but exhibits lower performance per token since the model must simultaneously attend to all five tool schemas and disambiguation instructions. The dual-agent architecture decomposes the context, focusing each pass on a single subtask to achieve higher accuracy-per-token.

7.6 Validation Layer Impact

The validation layer (Table 6) contributes modest gains, primarily correcting category casing and normalisation. This reveals that argument extraction remains the primary bottleneck, since validation cannot recover missing parameters (`PARAM_MISSING`).

8 Discussion

Privacy through Task Decomposition. The dual-agent architecture improves accuracy and makes local inference viable. By separating routing (single-digit output) from extraction (focused

Mode	Model	FSP	ACS	AVC	Task Acc	CR	Latency (s)	RAM (GB)
Single	EXAONE-4.0-1.2B	19.3	—	0.3	8.9	0.3	0.1	2.0
	Gemma-3-1B-it	27.0	—	47.5	30.0	0.9	0.8	1.6
	LFM2.5-1.2B	28.7	—	11.7	17.2	3.1	7.8	1.4
	MiniCPM5-1B	71.1	—	86.7	62.3	13.0	2.6	1.5
	Qwen3.5-0.8B	95.0	—	91.2	79.8	40.2	5.8	1.3
	Qwen3.5-2B	**96.5**	—	**97.6**	**92.0**	60.2	12.0	2.1
Dual	EXAONE-4.0-1.2B	90.6	3.7	3.3	37.8	3.0	5.0	1.9
	Gemma-3-1B-it	55.1	59.9	71.6	51.7	16.5	3.1	1.5
	LFM2.5-1.2B	69.6	51.6	57.2	51.8	16.3	4.1	1.4
	MiniCPM5-1B	84.9	77.0	81.6	74.3	43.5	3.9	1.5
	Qwen3.5-0.8B	**94.3**	90.2	96.8	89.1	61.7	3.0	1.2
	Qwen3.5-2B	92.0	91.6	**97.4**	90.4	**77.6**	6.2	2.1

Table 4: Main results (validated metrics, %). **Bold** = best per metric. FSP = Function Selection Precision; ACS = Argument Completeness Score; AVC = Argument Value Correctness; Task Acc = F_1-based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (—) for single-agent configurations where the joint output format makes isolated argument coverage less meaningful. Latency and RAM are per-query averages.

Group	Qwen3.5-0.8B T.Acc	Qwen3.5-0.8B CR	Qwen3.5-2B T.Acc	Qwen3.5-2B CR
Time Series	81.6	54.8	74.4	60.9
Distribution	89.6	56.2	93.8	79.0
Comparison	92.5	71.4	93.7	73.6
Calc. Total	87.9	59.1	91.4	84.5
Top Expenses	93.3	64.8	98.9	92.4

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

Model (Dual)	ΔTask Acc	ΔCR
EXAONE-4.0	+0.00	+0.00
Gemma-3-1B-it	+0.12	+0.00
LFM2.5-1.2B	+0.50	+0.00
MiniCPM5-1B	+1.17	+0.87
Qwen3.5-0.8B	+0.41	+1.57
Qwen3.5-2B	+0.29	+0.87

Table 6: Validation layer impact (Δ in percentage points, dual mode). Gains are modest but consistent for capable models.

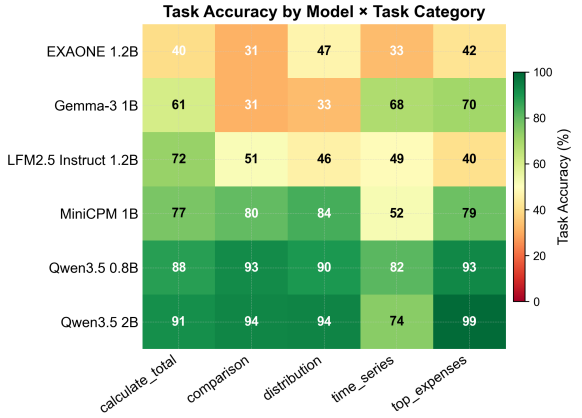


Figure 3: Task Accuracy (%) by Model × Tool Category in dual mode. Qwen3.5 models achieve >74% across all categories, while weaker models show marked gaps in comparison and time series tasks.

JSON), the dual-agent setup reduces the generation complexity per call, addressing the primary bottleneck of small models. Decomposing the task helps sub-2B models perform reliably on domain-specific schemas and by doing so, makes on-device inference a practical privacy guarantee rather than an aspirational one.

Metric-Specific Trade-offs. We note that the dual-agent advantage is not uniform across metrics. Optimised single-agent Qwen3.5-2B achieves the highest Task Accuracy (92.0% vs. 90.4% dual), because its joint prompt gives the model access to all tool schemas simultaneously, enabling cross-tool reasoning. However, this advantage comes at a cost: single-agent produces 22.6% `PARAM_EXTRA` errors (vs. 1.9% dual), which are silent corruptions in financial contexts. The dual-agent architecture is therefore preferred not because it uniformly outperforms single-agent, but because it delivers higher *strict* correctness (Correct Ratio: 77.6% vs. 60.2%), safer error modes, and substantially lower latency (6.2 s vs. 12.0 s for Qwen3.5-2B)—properties that matter more for user-facing financial applications.

Router vs. Simpler Intent Classifiers. To validate the LLM-based router, we benchmarked a lightweight embedding-similarity baseline using `all-MiniLM-L6-v2` (23M parameters). Cosine similarity between query embeddings and tool-description embeddings achieves 75.7% tool-selection accuracy (87/115), compared with ≥90% for the LLM router across all Qwen3.5 configurations. The embedding baseline's errors concentrate on `plot_time_series` (52.2% per-tool

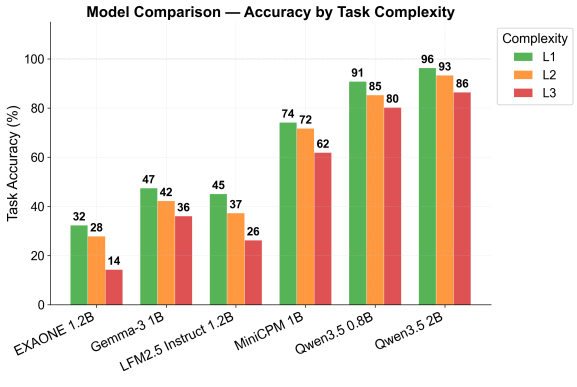


Figure 4: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

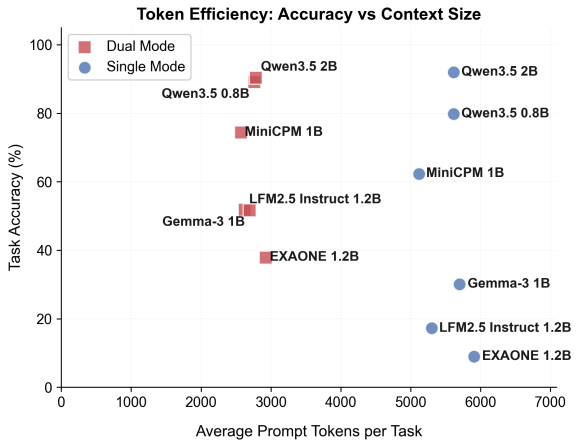


Figure 5: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue), demonstrating superior context utilisation.

accuracy), which it systematically confuses with `plot_comparison_bars` when queries mention date ranges—a contextual disambiguation task ("from 2024 to 2025" denotes a *range* for time series, not a *comparison* of two periods) that requires pragmatic reasoning beyond lexical overlap. Accuracy also degrades with query complexity (L1: 83.3%, L2: 75.6%, L3: 71.7%), confirming that an LLM-based router is warranted for our domain.

Absence of Cloud LLM Baselines. We omit cloud-hosted baselines (GPT-4, Claude) because transmitting expense categories in the prompt would expose user spending patterns to third parties, violating our privacy model. Synthetic-schema cloud comparisons to quantify privacy–accuracy trade-offs are left for future work.

Addressing Highly Complex Queries. Even the best model only achieves 59.6% strict Correct Ratio on L3 queries in dual mode, which involves complex cross-year range comparisons requiring up to 10 interdependent parameters. Fine-tuning on domain-specific examples using verified synthetic data (Liu et al., 2024b) could be a potential method to improve performance on complex queries.

Deployment Recommendations. Based on our evaluation, we recommend: **(1)** *Memory-constrained* (≤2 GB available): Qwen3.5-0.8B dual, 89.1% Task Acc, 3.0 s, 1.2 GB RAM. **(2)** *Accuracy-first*: Qwen3.5-2B dual, 90.4% Task Acc, 77.6% CR, 6.2 s, 2.1 GB RAM. **(3)** *Budget hardware*: MiniCPM5-1B dual, 74.3% Task Acc, 43.5% CR, 3.9 s, 1.5 GB RAM. All configurations keep user data entirely on-device.

Case Study. Figure 6 shows an authentic usage session of ExpenseSense. As a real-world deployment test in February 2026, an author used the system to query their own personal finance data in informal language-"can ya plot spending on snacks for past X months?". The dual-agent Qwen3.5-0.8B pipeline correctly routed this to `plot_time_series` and extracted the parameters `{category: snacks, months: 14}` - all in 6 seconds. As shown in the plot, the resulting visualization revealed a clear trend in weekly snack expenditure. By surfacing this data locally, the author was able to reflect on their habits and successfully reduce their spending over time (from peaks of ¥3,000–4,000 to under ¥1,000). Crucially, this entire interaction- query, inference, and plotting - remained strictly on the user's device; no sensitive data was transmitted externally at any point. Connecting technical mechanisms to human needs, this demonstrates how private, local analytics can directly support personal budgeting workflows.

9 Conclusion

We presented ExpenseSense, a human-centered evaluation of privacy-preserving, on-device financial tool calling using sub-2B quantized models. Our central finding is that local inference at this scale is not just technically feasible but practically reliable: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy with 3.0 s latency, keeping all user data—queries, context, and results—strictly on-device. Architectural decomposition into intent routing and parameter extraction further improves strict correctness (from 60.2% to 77.6% Correct Ratio)—though single-agent Qwen3.5-2B

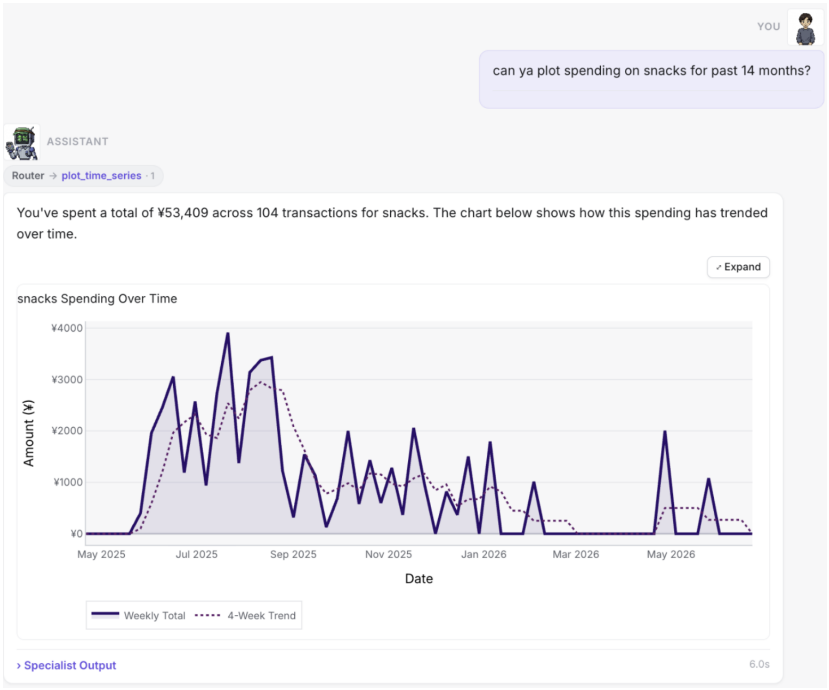


Figure 6: Real usage session: a user queries snack spending trends in informal language. The dual-agent Qwen3.5-0.8B pipeline correctly invokes `plot_time_series`, producing an actionable spending trend—entirely on-device with no data transmitted externally.

retains a slight Task Accuracy edge (92.0% vs. 90.4%)—and, critically, shifts error modes toward safer failures: spurious parameter additions, which can silently corrupt financial analytics and mislead user decisions, are replaced by missing-parameter errors that are more visible and less harmful.

Several directions warrant future investigation: (1) *Fine-tuning* on domain-specific function-calling data to address residual L3 failures. (2) Extending to *multi-turn* interactions with conversational context. (3) Conducting *user studies* on trust and mental models of on-device privacy. (4) Adding *multilingual* query support. (5) Evaluating generalisability to other *privacy-sensitive domains* (e.g., healthcare, legal). (6) Controlled comparisons against cloud LLMs using synthetic schemas to quantify privacy–accuracy trade-offs. (7) Testing lower-bit quantization (Q4_K_M) for mobile deployment.

Limitations & Ethics

- Quantisation is fixed at Q8_0; lower-bit formats (Q4_K_M) may yield different tradeoffs.
- Test cases were single-developer authored in English; crowdsourcing and multilingual data would improve diversity.
- Ground-truth labels are deterministic; some queries admit alternative valid tool calls.
- Privacy is architectural (no external data flow) rather than cryptographic.
- Cloud LLMs were not benchmarked as cloud transmission violates the privacy model; evaluation is also limited to five tools in personal finance.

Ethics Statement

ExpenseSense is designed for on-device inference specifically to preserve user financial privacy. No real user financial data was used in the benchmark; all test cases reference synthetic expense categories. The system is intended to complement, not replace, professional financial advice. We recognise that even on-device models may produce incorrect financial summaries, and advocate for clear user-facing disclaimers in production deployments.

Data and Code Availability

Code, prompts, benchmark data, and full results will be released on GitHub upon acceptance. Released artefacts include: all 115 test cases with ground-truth labels, the complete router and specialist prompt templates, inference configuration details, raw and validated CSV result files, and the benchmark runner, plotting scripts, and the exact model GGUFs used in this study to ensure complete reproducibility.

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Borda, Ashley Loren Bucher, and Sébastien Bubeck. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Yupeng Cao, Haohang Li, Weijin Liu, Wenbo Cao, Anke Xu, Lingfei Qian, Xiaozhe Peng, Yuxuan Tang, Zeyi Yao, Heng Huang, K. P. Subbalakshmi, Xiao Zhu, Jordan W. Suchow, and Yue Yu. 2026. FinTrace: Holistic trajectory-level evaluation of LLM tool calling for long-horizon financial tasks. *arXiv preprint arXiv:2604.10015*.

Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. *arXiv preprint arXiv:2404.01744*.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711.

B. Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.

Georgi Gerganov. 2023. llama.cpp.

Tom Gunter, Zirui Wang, Chong Wang, Ruoming Huang, Yandong Zhang, Xianzhi Liang, and Apple Intelligence Team. 2024. Apple intelligence foundation language models. *arXiv preprint arXiv:2407.21075*.

Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. Small models, big tasks: An exploratory empirical study on small language models for function calling. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 1117–1126.

LG AI Research. 2025. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. *arXiv preprint arXiv:2507.11407*.

Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Zhang, and Yunxin Liu. 2024. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.

Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Ying Liu, and Xiaojun Hu. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587*.

Liquid AI. 2026. LFM2.5 technical report. *Liquid AI Blog*.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024a. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32431–32454.

Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Tan, Joey Tianyi Zhou, Philip S. Yu, and Steven C. H. Hoi. 2024b. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37.

Helen Nissenbaum. 2004. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158.

OpenBMB. 2026. MiniCPM5-1B: A SOTA 1B on-device LLM.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 48371–48392.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Siyuan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, and Zhiyuan Liu. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents. Qwen Blog.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, volume 36.

Yewei Song, Xunzhu Tang, Cedric Lothritz, Saad Ezzini, Jacques Klein, Tegawendé F. Bissyandé, Andrey Boytsov, Ulrick Ble, and Anne Goujon. 2025. CallNavi: A challenge and empirical study on LLM function calling and routing. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*.

Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. 2024. Beyond memorization: Violating privacy via inference with large language models. In *Proceedings of the 12th International Conference on Learning Representations*.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. 2023. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*.

Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*.

Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. FinGPT: Open-source financial large language models. In *International Symposium on Large Language Models for Financial Services (FinLLM 2023) at IJCAI 2023*.

Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211.

Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng Chen, Yong Liu, and Chi Zhang. 2026. FinMCP-Bench: Benchmarking LLM agents for real-world financial tool use under the model context protocol. *arXiv preprint arXiv:2603.24943*.

A Complexity Scoring Features

Feature	Range	Description
n_p	$0+$	Number of expected parameters
d_{date}	$0-4$	Date complexity: none / year / month / range / specific day
d_{cat}	$0-2$	Category distance: exact / alias / semantic
d_{rel}	$0-1$	Relative time expression ("past X months")
d_{multi}	$0-2$	Compound parameter groups
d_{abbr}	$0-1$	Abbreviated year formats ("'24")
d_{hol}	$0-2$	Holiday/calendar knowledge required

Table 7: Seven features of the complexity scoring function (Eq. 1).

B Known Category Vocabulary

The post-hoc parameter validation layer (Section 4.3) and the LLM prompts rely on a known category vocabulary defined dynamically or in static schema files. Table 8 lists the 11 major categories defined in the application alongside the mapped subcategories/aliases from the system configuration (`categories.json`) that are passed in the `{metadata}` block.

Major Category	Mapped Minor Categories / Subcategories
Housing and Utilities	housing, internet bill, electricity bill, gas bill, water & sewage bill, phone bill
Food	grocery, snacks, cafe, café, coffee, bento, beverage, combini meal, dining, eating with friend
Transportation	commute, ride share, tokyo metro, Tokyo Metro, flight tickets, cable car, bus, shinkansen, car rental, taxi
Accommodation	stay
Fitness	supplements, shoes, sports event, sports watch, sports clothing, sports rental, gym, sports equipment, basketball game, football game, futsal game
Souvenirs/Gifts/Treats	souvenirs, treat, gift, souvenirs/gifts/treats
Household and Clothing	clothing, household
Entertainment	entertainment, nomikai, activities, Activities, arcades & karaoke, Arcades & Karaoke, events & venues, Events & Venues
Miscellaneous	medicines, personal care, misc, help, charity, donation, entrance fees, park entrance fees, healthcare
Education	tuition, books, exam fees, notebooks, textbook
Electronics and Furniture	furniture, electronics

Table 8: Known category vocabulary from `categories.json`.

C Sample Test Cases

D Qualitative Error Examples

Table 10 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

ID	Query	Tool	Expected Params	Tier
TS01	Can ya show spending on food for past 6 months?	`plot_time_series`	category=Food, months=6	L2
DI06	show breakdown of expenses for june 21 2024	`plot_distribution`	year=2024, month=6, day=21	L2
CP24	compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026	`plot_comparison_bars`	category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4	L3
CT07	sum total spent on metro for july 2024?	`calculate_total`	category=tokyo metro, year=2024, month=7	L2
TP06	top 5 expenses past month? disregard rent	`get_top_expenses`	n=5, months=1, ignore_rent=true	L2

Table 9: Sample test cases. Note informal language, abbreviations, and semantic mappings ("metro"→"tokyo metro", "groceries"→"grocery").

Error Type	Query	Expected	Model Output
PARAM_MISSING	"Show spending trend for the last 3 months"	{months: 3}	{} — `months` omitted in the relative-time case
PARAM_VALUE_WRONG	"compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026"	{em1: 4, em2: 4}	{em1: 11, em2: 11} — confuses start month for end month in cross-year range
PARAM_EXTRA	"How much did I spend on food?"	{category: Food}	{category: Food, months: 1} — spurious time filter restricts scope to last month, silently returning an incorrect total

Table 10: Representative failure cases for Qwen3.5-2B (dual mode). `PARAM_EXTRA` is the most dangerous failure mode in financial contexts: the spurious `months=1` filter silently restricts "total food spending" to "last month's food spending", returning a plausible but incorrect figure that a user may act on without realising the scope was changed.

E Supplementary Figures

Figure 7 provides the routing confusion matrix across tools, while Figure 8 shows the per-category footprint across all evaluated models. Figure 9 illustrates the metrics improvement provided by the post-hoc validation layer.

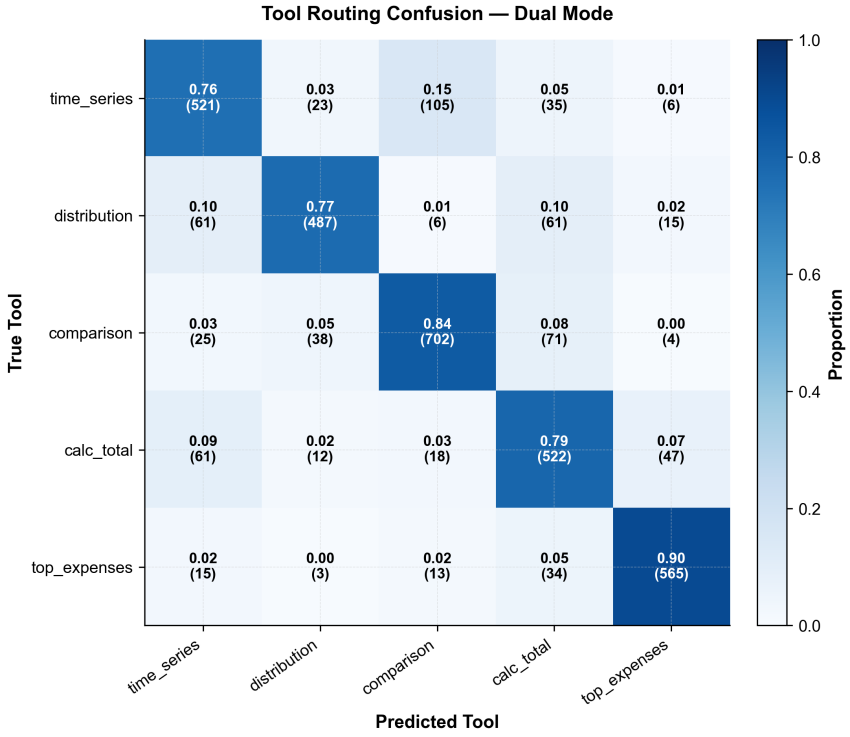


Figure 7: Normalized routing confusion matrix for the dual-agent router. The moderately high diagonal accuracy (76–90%) validates that the sequential classification step introduces negligible routing error propagation.

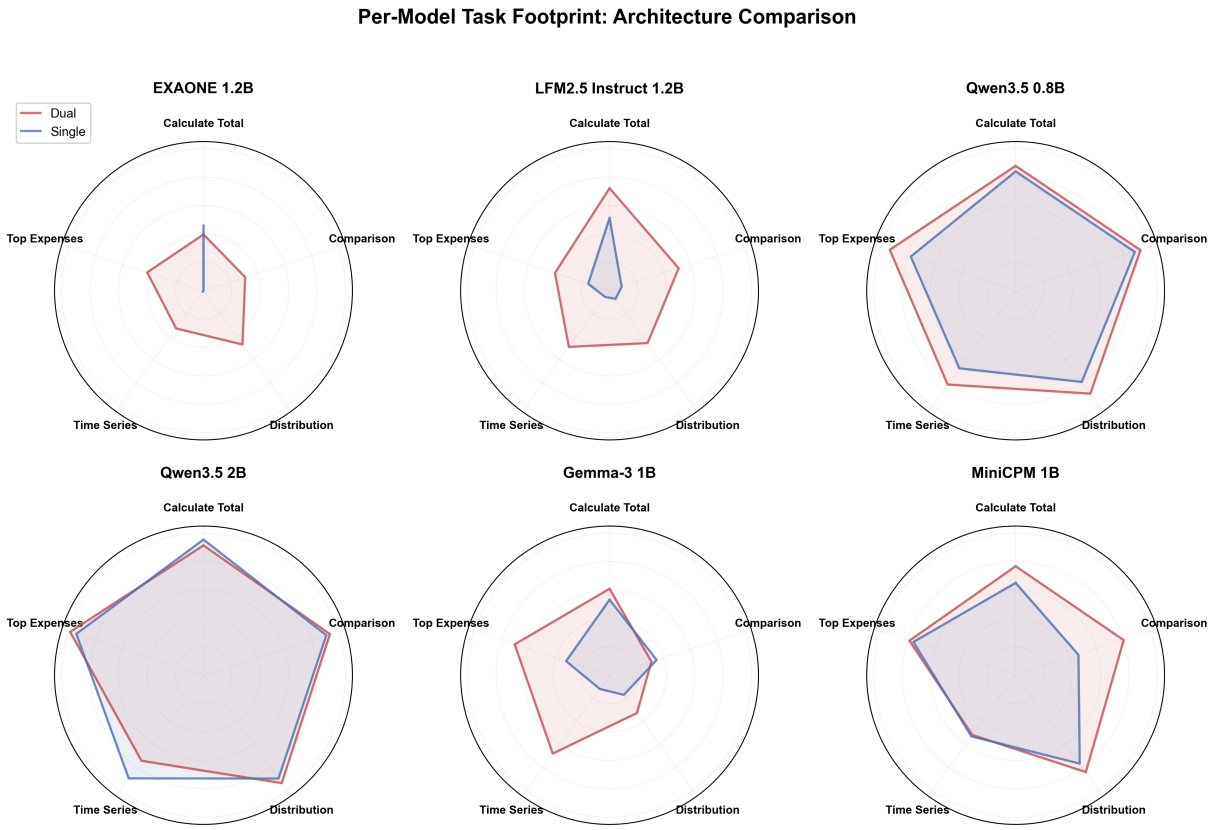


Figure 8: Per-model task footprint across all five task categories (Single vs. Dual mode). The dual-agent architecture provides near-universal improvement across models and tasks.

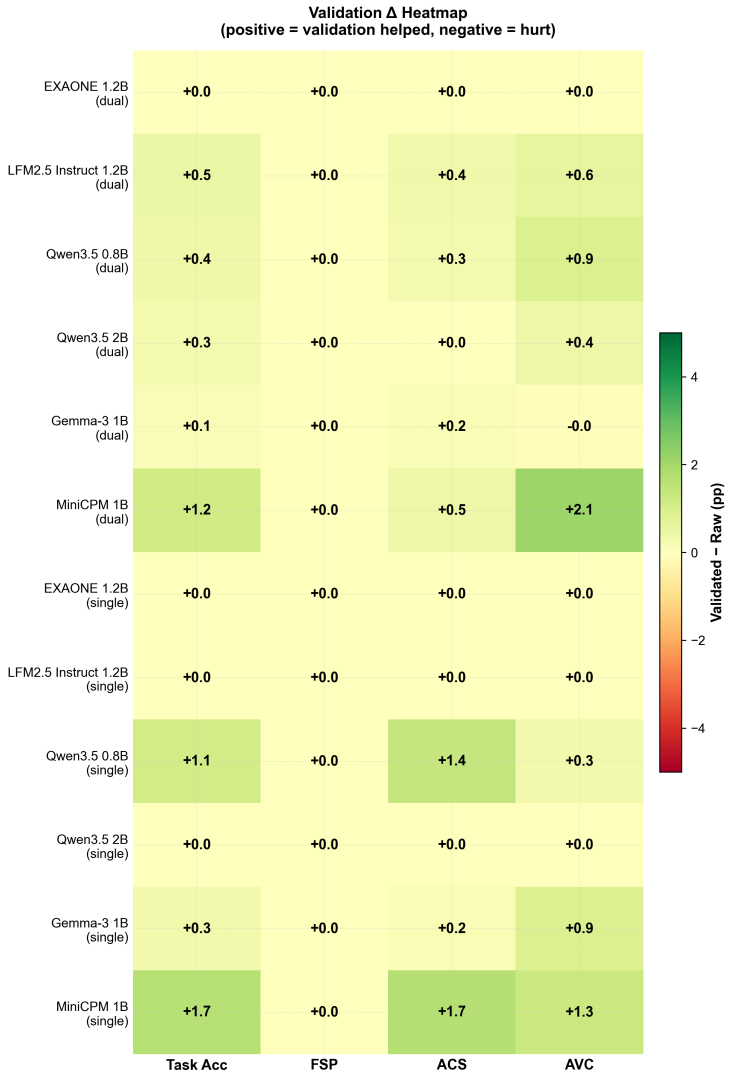


Figure 9: Validation Δ Heatmap, showing absolute percentage point improvement across metrics after applying post-hoc fuzzy matching.

F Prompt Templates

The on-device prompts utilized for routing and parameter extraction tasks are detailed below.

F.1 Router Prompt

The router receives a focused intent-classification prompt:

```
You are an expert intent classifier for an expense analysis assistant.
Your job is to determine which ONE tool is best suited to answer the user's
question.

## Available Tools

1. **`plot_time_series`** (ID: 1)
   - Use when user asks about: trends, spending over time, "past X months",
     "since 2023", date ranges.
   - Keywords: trend, over time, plot, show spending, changed, since.
   - NOTE: if user says "trend" or "over time" WITH "exclude rent", still
     use this tool.

2. **`plot_distribution`** (ID: 2)
   - Use when user asks for: breakdown, distribution, proportions, pie
     chart, "how is X split", "what does X look like".
   - Keywords: breakdown, distribution, distro, pie chart, split, look like.
   - NOTE: if user says "breakdown" or "split" WITH "exclude rent", still
     use this tool.

3. **`plot_comparison_bars`** (ID: 3)
   - Use ONLY when comparing TWO different time periods (e.g. "Dec 2024 vs
     Dec 2025" or ranges like "Jan-Jun 2024 vs Jan-Jun 2025").
   - REQUIRES two distinct periods. If the user only mentions one month or
     one year, DO NOT use this tool.
   - Keywords: compare, contrast, vs, versus, difference between.

4. **`calculate_total`** (ID: 4)
   - Use when user asks for: simple totals, sums, specific amounts "how
     much did I spend".
   - Keywords: how much, total, sum, cost.

5. **`get_top_expenses`** (ID: 5)
   - Use when user asks for: biggest/largest expenses, top X items, most
     expensive.
   - Keywords: biggest, largest, top, most expensive, highest.
   - NOTE: if user says "top X" WITH "exclude rent", still use this tool.

## Output Format
Output ONLY the tool number (1-5). Do not output anything else, NO
MARKDOWN, NO BACKTICKS, NO EXPLANATION, NO CODE!

## WRONG outputs (NEVER do this):
[X] I think the best tool is 4
[X] `4`
[X] Based on the query, I would use 1 because...
[X] ```4```
[X] The answer is: 2

## CORRECT outputs:
[v] 4
[v] 1

## Examples
User: "How much did I spend on food?"
Output: 4

User: "Total cost of internet for the last 3 months"
Output: 4

User: "How much did I spend in the last 6 months without rent?"
Output: 4

User: "Show me my spending trend for groceries"
Output: 1

User: "How has my rent changed over the last 6 months?"
Output: 1

User: "Show spending trend for past 4 months exclude rent tho"
Output: 1

User: "Compare food spending in 2024 and 2025"
Output: 3

User: "Difference in spending between Dec 2023 and Dec 2024"
Output: 3

User: "Compare overall spend in '24 vs '25 (exclude rent tho)"
```

```
Output: 3

User: "Compare dining between Jan-Jun 2024 and Jan-Jun 2025"
Output: 3

User: "Contrast spending from Nov 2024 to Feb 2025 vs Nov 2025 to Feb 2026"
Output: 3

User: "What were my biggest expenses?"
Output: 5

User: "Show me my top 5 largest purchases this year"
Output: 5

User: "Top 10 items without rent"
Output: 5

User: "Pie chart of transportation"
Output: 2

User: "How is my spending split in 2023?"
Output: 2

User: "Show me my spending breakdown excluding rent for the past 6 months"
Output: 2

User: "what does my electronics spending look like for jan 2025?"
Output: 2

User: "share distro of household expenses"
Output: 2
```

F.2 Specialist Base Template

Each specialist prompt instantiates a shared template defining output structure and relative date handling. The `{metadata}` placeholder is populated dynamically with the categories present in the user-uploaded dataset:

```
You are an automated API parameter extractor. Your ONLY purpose is to extract
parameters from the user's request and output a SINGLE JSON object containing
those parameters.

## CRITICAL RULES (READ CAREFULLY)
1. DO NOT write any Python code or function calls. DO NOT import anything.
2. DO NOT wrap the output in markdown. NO backticks (```).
3. Output EXACTLY and ONLY a JSON object representing the parameters.
   E.g. {"category": "Food", "months": 6}
4. Use EXACT category names from the metadata below. If not an exact match,
   map it to the closest valid category (e.g. 'groceries' -> 'grocery',
   'dining out' -> 'dining').

## Date Rules
- `months=N`: RELATIVE duration (e.g. "past month" -> months=1, "last 3
  months" -> months=3, "past year" -> months=12).
- Fractional years: convert to months. "past 0.5 years" -> months=6, "past
  2.5 years" -> months=30.
- Abbreviated years: '24 = 2024, 24 = 2024, ALWAYS output the full 4-digit
  year.
- `year=YYYY`: specific full calendar year.
- `year=YYYY, month=M`: specific calendar month (e.g. "Jan 2024").
- `year=YYYY, month=M, day=D`: specific calendar date.
- `start_year=YYYY, end_year=YYYY`: specific year range.
- `start_year=YYYY, start_month=M, end_year=YYYY, end_month=M`: specific
  month-to-month range.
- For comparing two ranges (e.g., comparing Jan-Jun 2024 vs Jan-Jun 2025),
  use comparison range parameters: `y1=2024, sm1=1, em1=6, y2=2025, sm2=1,
  em2=6`.
- For comparing cross-year ranges (e.g., Nov 2024-Jan 2025 vs Nov 2025-Jan
  2026), use `ey1` and `ey2`: `y1=2024, sm1=11, ey1=2025, em1=1, y2=2025,
  sm2=11, ey2=2026, em2=1`.
- CRITICAL: For relative queries like "past month", "last month", "last 6
  months", ALWAYS use `months=N`. Do NOT use `month=M`.
- Pick ONLY ONE time filter. Do not mix `months` with `year`.

## Available Parameters
{parameters}

## Examples
{examples}

## Context
```
{metadata}
```
Currency: JPY
Today: {current_date}
```

```
FINAL REMINDER: Output ONLY the JSON object. NO MARKDOWN, NO BACKTICKS,
NO EXPLANATION, NO CODE!
```

F.3 Specialist Tool Prompts

Each tool-specific specialist appends its unique parameter definitions and few-shot examples to the base template. The precise specifications for each tool are detailed below.

F.3.1 plot_time_series

```
Parameters:
  category (str, optional), year (int, optional), month (int, optional),
  start_year (int, optional), start_month (int, optional),
  end_year (int, optional), end_month (int, optional),
  months (int, optional), ignore_rent (bool, optional, defaults to false)

## Examples
Q: "How much did I spend on futsal for the past 6 months?"
{"category": "futsal game", "months": 6}

Q: "Show me food spending from 2023 to 2025"
{"category": "Food", "start_year": 2023, "end_year": 2025}

Q: "Plot spending on snacks from oct 2024 to dec 2025"
{"category": "snacks", "start_year": 2024, "start_month": 10, "end_year": 2025, "end_month": 12}

Q: "Can ya plot spending on futsal on dec 2024?"
{"category": "futsal game", "year": 2024, "month": 12}

Q: "Gym expenses in 2024?"
{"category": "gym", "year": 2024}

Q: "Show me my spending trend for the last 3 months"
{"months": 3}

Q: "plot spending from june 2023 to dec 2025"
{"start_year": 2023, "start_month": 6, "end_year": 2025, "end_month": 12}

Q: "Trend of electricity bills since 2022"
{"category": "electricity bill", "start_year": 2022}

Q: "How has my transportation spending changed over time?"
{"category": "Transportation"}

Q: "Show spending on combinis for past year"
{"category": "combini meal", "months": 12}

Q: "Show spending trend for past 4 months exclude rent tho"
{"months": 4, "ignore_rent": true}

Q: "plot spend on cafes for past 2.5 yrs"
{"category": "cafe", "months": 30}

Q: "show my ride share costs from '24 to 26"
{"category": "ride share", "start_year": 2024, "end_year": 2026}

Q: "show electronics spending since 2023"
{"category": "electronics", "start_year": 2023}
```

F.3.2 plot_distribution

```
Parameters:
  category (str, optional), year (int, optional), month (int, optional),
  day (int, optional), start_year (int, optional), start_month (int, optional),
  end_year (int, optional), end_month (int, optional), months (int, optional),
  ignore_rent (bool, optional, defaults to false)

## Examples
Q: "Show me a breakdown of my food expenses in 2024"
{"category": "Food", "year": 2024}

Q: "Pie chart of all expenses from Nov 2024 to Feb 2025"
{"start_year": 2024, "start_month": 11, "end_year": 2025, "end_month": 2}

Q: "Show me my spending breakdown for last month"
{"months": 1}

Q: "Show me my spending breakdown for groceries for past 6 months"
{"category": "Grocery", "months": 6}

Q: "show spend distribution for 2026 feb"
{"year": 2026, "month": 2}

Q: "show breakdown of all expenses from 2024 to 2025"
{"start_year": 2024, "end_year": 2025}
```

```
Q: "Show me a breakdown of expenses for Dec 2024 (exclude rent tho)"
{"year": 2024, "month": 12, "ignore_rent": true}

Q: "Show spending breakdown for 2023/06/22"
{"year": 2023, "month": 6, "day": 22}

Q: "Spending distribution for the last 3 months"
{"months": 3}

Q: "Breakdown of transportation spending from 2022 to 2024"
{"category": "Transportation", "start_year": 2022, "end_year": 2024}

Q: "How did I split my money between categories in 2023?"
{"year": 2023}

Q: "Show me my spending breakdown excluding rent for the past 6 months"
{"months": 6, "ignore_rent": true}

Q: "Breakdown of fitness spending for 2025"
{"category": "Fitness", "year": 2025}

Q: "Breakdown of my spending since 2024"
{"start_year": 2024}

Q: "show breakdown of all expenses for mar 2026 (exclude rent tho)"
{"year": 2026, "month": 3, "ignore_rent": true}

Q: "what does my electronics and furniture spending look like for jan 2025?"
{"category": "Electronics and Furniture", "year": 2025, "month": 1}
```

F.3.3 plot_comparison_bars

```
Parameters:
  category (str, optional), y1 (int, optional), m1 (int, optional),
  d1 (int, optional), y2 (int, optional), m2 (int, optional),
  d2 (int, optional), sm1 (int, optional), em1 (int, optional),
  sm2 (int, optional), em2 (int, optional), ey1 (int, optional),
  ey2 (int, optional), ignore_rent (bool, optional, defaults to false)

## Examples
Q: "Compare food spending in 2024 vs 2025"
{"category": "Food", "y1": 2024, "y2": 2025}

Q: "Compare dining Jan 2024 vs Jan 2025"
{"category": "dining", "y1": 2024, "m1": 1, "y2": 2025, "m2": 1}

Q: "How does my spend on snacks compare between jan 2024 and jan 2026?"
{"category": "snacks", "y1": 2024, "m1": 1, "y2": 2026, "m2": 1}

Q: "Compare total spending 2022 vs 2023"
{"y1": 2022, "y2": 2023}

Q: "Compare overall expenses from jan 2025 vs jan 2026"
{"y1": 2025, "m1": 1, "y2": 2026, "m2": 1}

Q: "Compare transportation on 21 July 2024 vs 21 July 2025"
{"category": "Transportation", "y1": 2024, "m1": 7, "d1": 21, "y2": 2025, "m2": 7, "d2": 21}

Q: "Compare spend on electricity 2024 vs 2025"
{"category": "electricity bill", "y1": 2024, "y2": 2025}

Q: "Compare gym spending Nov 2024 vs Nov 2025"
{"category": "gym", "y1": 2024, "m1": 11, "y2": 2025, "m2": 11}

Q: "Compare my spending in 2024 vs 2025 excluding rent"
{"y1": 2024, "y2": 2025, "ignore_rent": true}

Q: "compare education expenses '24 vs '25"
{"category": "Education", "y1": 2024, "y2": 2025}

Q: "compare household expenses for april '24 vs apr 2025"
{"category": "household", "y1": 2024, "m1": 4, "y2": 2025, "m2": 4}

Q: "Compare dining between Jan-Jun 2024 and Jan-Jun 2025"
{"category": "dining", "y1": 2024, "sm1": 1, "em1": 6, "y2": 2025, "sm2": 1, "em2": 6}

Q: "compare electricity from jan-mar '24 vs jan-mar '25"
{"category": "electricity bill", "y1": 2024, "sm1": 1, "em1": 3, "y2": 2025, "sm2": 1, "em2": 3}

Q: "Contrast overall spend from Nov 2024 to Feb 2025 vs Nov 2025 to Feb 2026"
{"y1": 2024, "sm1": 11, "ey1": 2025, "em1": 2, "y2": 2025, "sm2": 11, "ey2": 2026, "em2": 2}

Q: "compare total spend on new years eve 2024 vs for same day on 2025"
{"y1": 2024, "m1": 12, "d1": 31, "y2": 2025, "m2": 12, "d2": 31}
```

F.3.4 calculate_total

```
Parameters:
```

```
  category (str, optional), year (int, optional), month (int, optional),
  day (int, optional), start_year (int, optional), start_month (int, optional),
  end_year (int, optional), end_month (int, optional), months (int, optional),
  ignore_rent (bool, optional, defaults to false)

## Examples
Q: "How much did I spend on groceries in Dec 2024?"
{"category": "grocery", "year": 2024, "month": 12}

Q: "Total spending from Oct 2024 to March 2025"
{"start_year": 2024, "start_month": 10, "end_year": 2025, "end_month": 3}

Q: "What is my total spending in 2025?"
{"year": 2025}

Q: "How much did I spend on food in past month?"
{"category": "Food", "months": 1}

Q: "Total cost of electricity in 2023"
{"category": "electricity bill", "year": 2023}

Q: "Total spent on 15 Jan 2024?"
{"year": 2024, "month": 1, "day": 15}

Q: "How much spent on rent in the last 6 months?"
{"category": "rent", "months": 6}

Q: "Sum of all transportation expenses in 2025"
{"category": "Transportation", "year": 2025}

Q: "Total spent on combini food for 2025?"
{"category": "combini meal", "year": 2025}

Q: "How much did I spend in the last 6 months without rent?"
{"months": 6, "ignore_rent": true}

Q: "Total spent on dining since 2024"
{"category": "dining", "start_year": 2024}

Q: "whats the total spend on donations from '23 to 2026?"
{"category": "donation", "start_year": 2023, "end_year": 2026}

Q: "Excluding rent, what was my total spend in 2025?"
{"year": 2025, "ignore_rent": true}
```

F.3.5 get_top_expenses

```
Parameters:
  n (int, optional, defaults to 10), category (str, optional),
  year (int, optional), month (int, optional), day (int, optional),
  start_year (int, optional), start_month (int, optional),
  end_year (int, optional), end_month (int, optional),
  months (int, optional), min_amount (int, optional),
  ignore_rent (bool, optional, defaults to false)

## Examples
Q: "What were my biggest expenses in Dec 2024?"
{"n": 10, "year": 2024, "month": 12}

Q: "Top 5 food expenses in 2024?"
{"n": 5, "category": "Food", "year": 2024}

Q: "Top 10 food expenses in june 2025?"
{"n": 10, "category": "Food", "year": 2025, "month": 6}

Q: "Top 5 expenses for 26 june 2024?"
{"n": 5, "year": 2024, "month": 6, "day": 26}

Q: "Top expenses from July 2023 to Dec 2024"
{"start_year": 2023, "start_month": 7, "end_year": 2024, "end_month": 12}

Q: "Show my top 10 expenses from 2023 to 2025"
{"n": 10, "start_year": 2023, "end_year": 2025}

Q: "Top 10 expenses of past month?"
{"n": 10, "months": 1}

Q: "What are my top expenses excluding rent?"
{"n": 10, "ignore_rent": true}

Q: "Show my top 3 largest transactions in 2023"
{"n": 3, "year": 2023}

Q: "What were the biggest expenses over the last 3 months, without rent?"
{"n": 10, "months": 3, "ignore_rent": true}

Q: "Top 7 futsal expenses for 2025?"
```

```
{"n": 7, "category": "futsal game", "year": 2025}

Q: "Top 5 expenses last month, exclude rent"
{"n": 5, "months": 1, "ignore_rent": true}

Q: "What were my biggest expenses since 2023?"
{"n": 10, "start_year": 2023}

Q: "show me top 10 expenses from jan 2025 to june ’25"
{"n": 10, "start_year": 2025, "start_month": 1, "end_year": 2025, "end_month": 6}

Q: "tell me the top 15 expenses for 2025, ignore rent plz"
{"n": 15, "year": 2025, "ignore_rent": true}
```